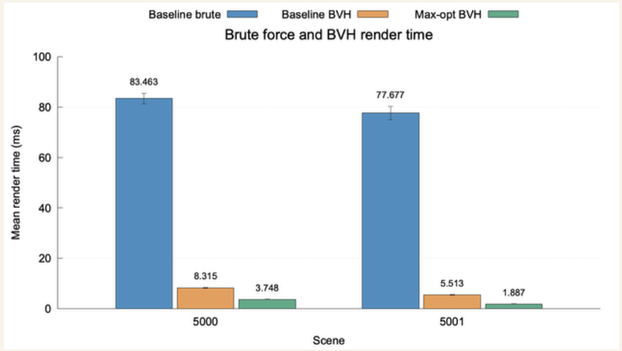
Optimized Ray Tracing

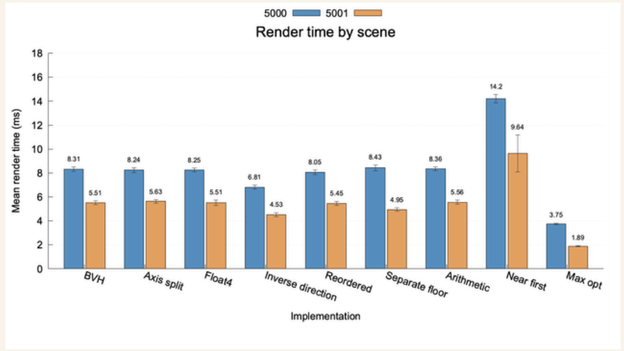
Adithya Kaushik

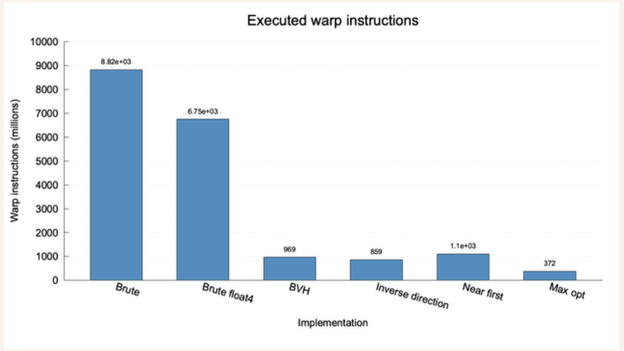
CS23B002

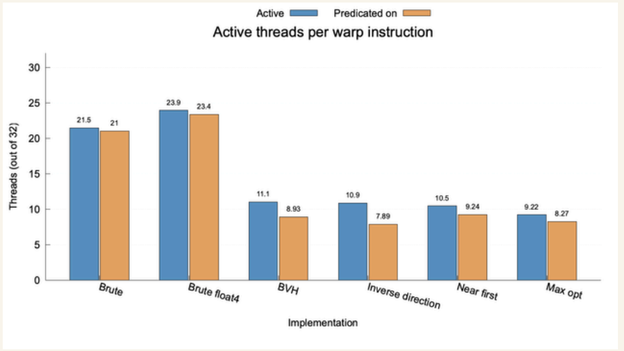
“you will build a working ray tracer in CUDA, add reflections and shadows, then make it faster.”

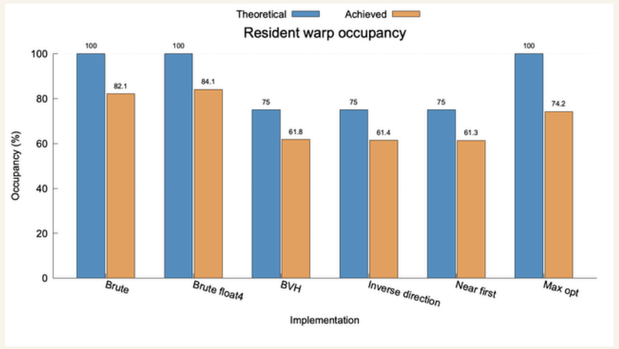
Images were compared against baseline byte for byte. Only -use_fast_math had differences

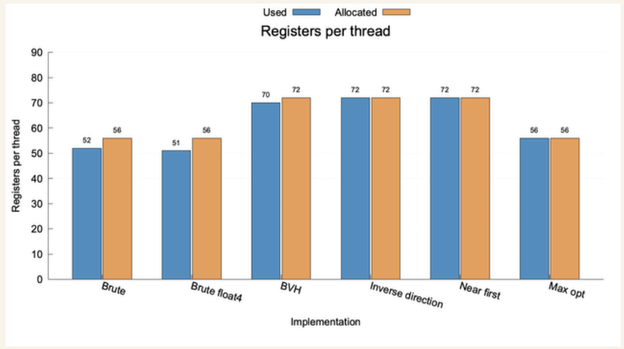


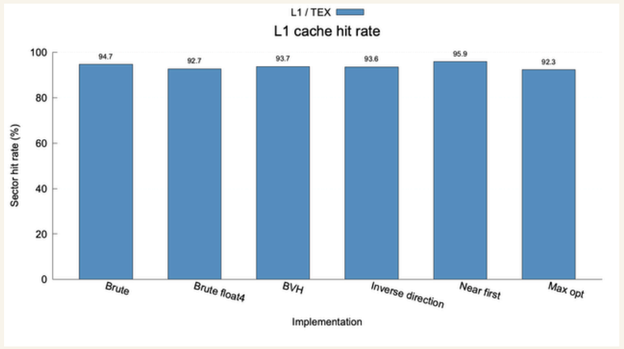


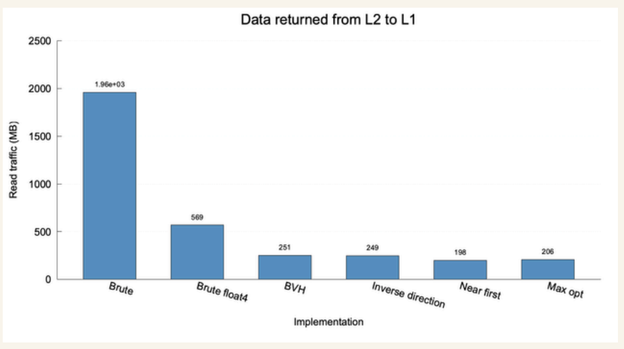




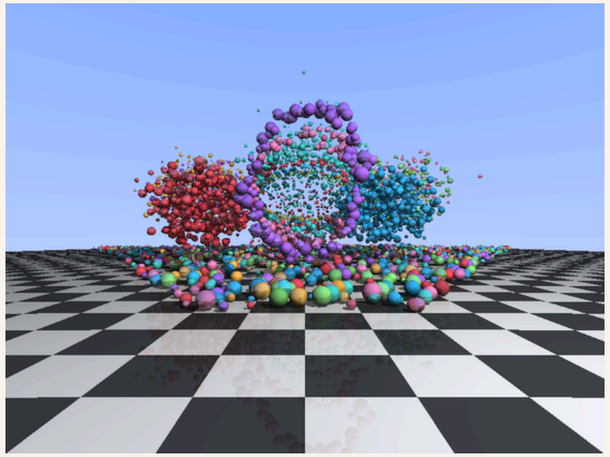


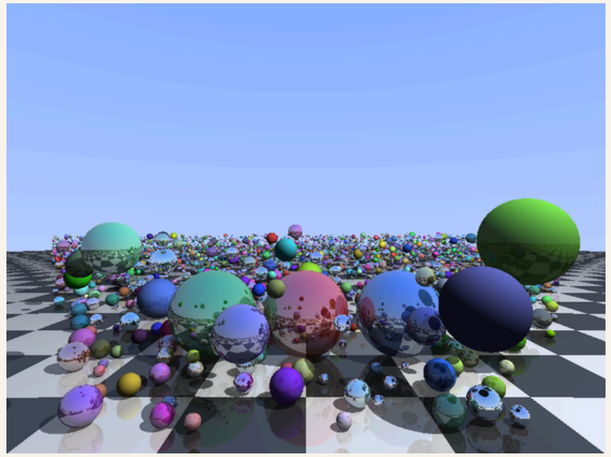


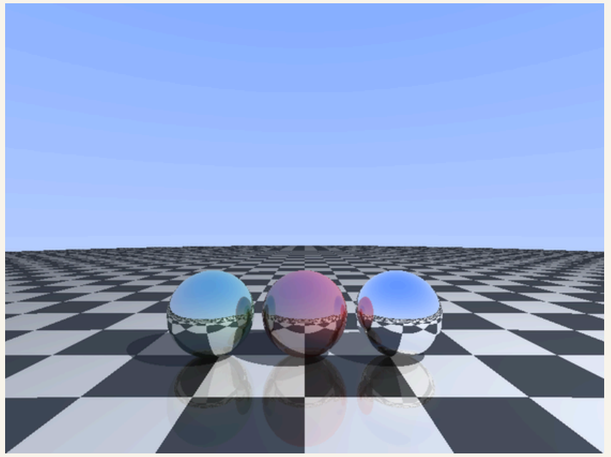




- *Smaller block sizes win on bigger scenes ~ 8x8*
- *Latency scales with number of pixels and bounce depth*
- *Best leaf size = 1*
- *~41x on 5001, ~22x on 5000*







Optimizations

- **BVH acceleration** : ***large improvement*** over brute force.
- **Brute-force float4 sphere attributes** : clear ***improvement*** on both large scenes. (sphere center and radius squared packed into float4)
- **Inverse ray direction reuse - *consistently improved BVH.***
- **BVH Splitting Heuristic** - changing from center extent median to ***AABB extent median helped a lot on bigger scenes***, probably since it accounted for sphere radius more concretely.

Successful only in some cases

- **Separate floor intersection** - helped scene 5001, not scene 5000.
- **-use_fast_math compiler flag** - useful on scene 5001, ***inconclusive on scene 5000.***
- **Constant memory float4 sphere attributes array** - helped brute force at 100 spheres, ***generally worsened larger cases.***

Failed/ inconclusive

- **Standalone near-first traversal** - ***substantially slower,*** I think due to overlaps in spheres and bounding boxes that skewed traversal.
- **Reordered scene storage** - small gains.
- **BVH float4 sphere attributes** - ***no clear improvement.***
- **Simplified sphere intersection arithmetic** - no clear standalone gain.
- **Reusing child AABB tests**, stacking only deferred children and pruning with cached entry distances - included in maximally optimized BVH code, but its individual contributions were not tested.

- Improving the BVHNode implementation's memory layout with 3 float4s covering all 12 min and max coordinates and one int4 for child references for AABB could possibly reduce loads and latency and also improve L1 cache hit rate, allowing more nodes to stay in cache. This was not properly tested.

Challenges

- **Reflection Colour Accumulation:** Initially considered maintaining a DFS-style stack of ray intersections up to the maximum bounce depth, but handling early termination when a ray reached the sky was difficult. Replaced this with an iterative colour accumulator that tracks the remaining reflectivity (residue), allowing colour contributions to be accumulated at each bounce.
- **Incorrect Near-First BVH Traversal:** Initially passed tClosest and tFarthest by value to intersectAABB(), preventing intersection distances from propagating to the caller. Corrected this by passing them by reference, enabling proper near-first traversal.
- **Floor Handling in BVH:** The large floor sphere produced oversized bounding boxes that adversely affected BVH performance. Excluded it from BVH construction while retaining separate intersection tests for primary, reflected, and shadow rays.
- **Ray–Sphere Intersection Arithmetic:** Initially mixed coefficients from the standard quadratic formula and the simplified half-(b) formulation, producing incorrect intersections. Corrected the coefficient definitions and root calculations while preserving support for both near and far intersections.